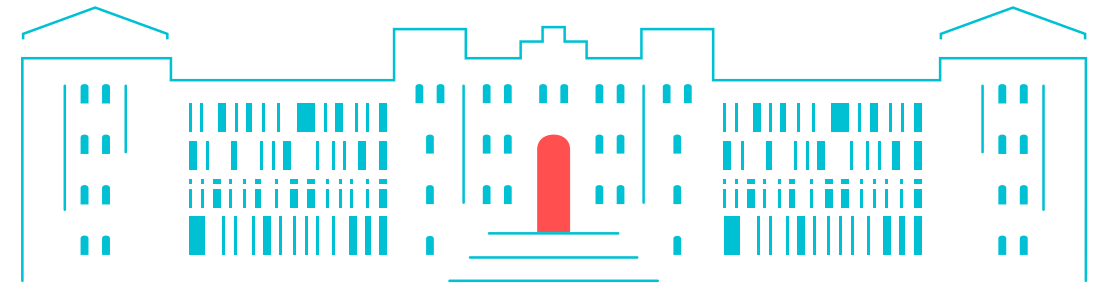
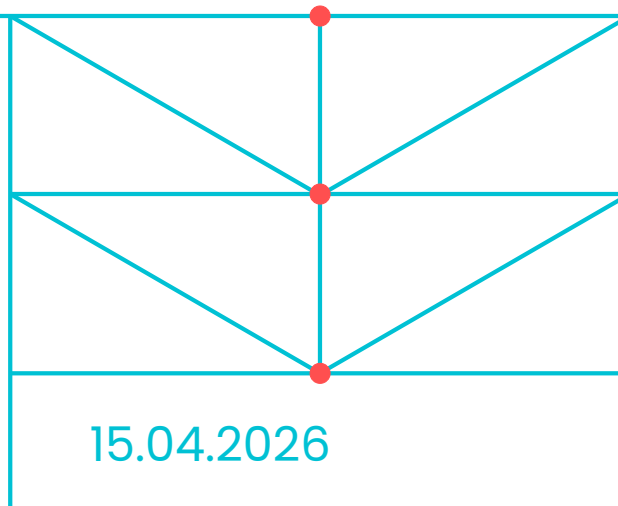


Parallelization of Solvers

Fynn Bense

Chair Computational Mathematics
Institute of Mathematics (E-10)
Building E, Room 3.045

TUHH
Hamburg
University of
Technology



02 – Basic OpenMP

- OpenMP: Directives, parallel constructs
- Race condition, Synchronization
- Data-sharing attributes
- Reduction
- Cache-coherence, False sharing

Additional reading

- Official resources: <https://www.openmp.org/resources/>
- Online Tutorial: <https://hpc-tutorials.llnl.gov/openmp/>

- **Fork – Join** programming model
- Thread – Team – Task

GNU C Compiler OpenMP

```
gcc -fopenmp ...
```

C runtime library definition

```
#include <omp.h>
```

- **Fork – Join** programming model
- Thread – Team – Task

GNU C Compiler OpenMP

```
gcc -fopenmp ...
```

C runtime library definition

```
#include <omp.h>
```

```
omp_get_num_threads()
```

Returns the number of threads that are currently in the team executing the parallel region from which it is called

```
omp_get_thread_num()
```

Returns the thread number of the thread, within the team, making this call

```
omp_set_num_threads()
```

Sets the number of threads that will be used in the next parallel region(s)

OMP_NUM_THREADS

Sets number of threads to use in the OpenMP program, i.e, `export OMP_NUM_THREADS={NUM}`

Directive format

```
#pragma omp directive-name [clause, ...] newline
```

- Directives are case-sensitive
- Only one directive-name can be specified per directive
- Each directive applies to the next statement which must be a structured block
- Clauses are optional and can be in any order
- Newline is required, and precedes the structured block which is enclosed by the directive.

```
1  #pragma omp parallel [clause ...] newline  
2      structured_block  
3
```

- Thread that reaches a parallel directive creates a team of threads and becomes the master of the team
- The code is duplicated and all threads in the team will execute the code contained in the structured block
- Inside the region threads are identified by consecutive numbers starting at zero
- There is an implied barrier at the end of a parallel section, only the master thread continues past this point → **recap:** Hello World OpenMP example

```
1  #pragma omp parallel [clause ...] newline
2      structured_block
3
```

- Thread that reaches a parallel directive creates a team of threads and becomes the master of the team
- The code is duplicated and all threads in the team will execute the code contained in the structured block
- Inside the region threads are identified by consecutive numbers starting at zero
- There is an implied barrier at the end of a parallel section, only the master thread continues past this point → **recap:** Hello World OpenMP example
- Optional clauses can be used to modify behavior and data environment of the parallel region

```
1 // Include the OpenMP header
2 #include <omp.h>
3
4 // This we only need to write output
5 #include <stdio.h>
6
7 int main(int argc, char** argv)
8 {
9     // Begin the parallel part of the code: here, the OpenMP library will spawn a number of threads equal
10    // to the OMP_NUM_THREADS environment variable. Each thread will run the code inside the brackets.
11    #pragma omp parallel
12    {
13
14        // This integer will store the number (rank) of the current thread
15        int thread_num;
16
17        // This integer will store the total number of threads
18        int num_threads;
19
20        // get the number of the current thread
21        thread_num = omp_get_thread_num();
22
23        // get the total number of threads
24        num_threads = omp_get_num_threads();
25
26        printf("Hello World... from thread number = %d out of %d many threads \n", thread_num, num_threads);
27    }
28    // Ending of parallel region
29
30    // Important: if you ask how many threads there are outside of the parallel region, the answer will be one!
31    printf("Number of threads outside of parallel region: %d \n", omp_get_num_threads());
32 }
```


Assumes that a parallel region has already been initiated, otherwise the construct executes in serial!

```
1  #pragma omp for [clause ...] newline
2  for_loop
3
```

Combined parallel construct

Abbreviation that corresponds to an individual parallel directive being immediately followed by a work sharing parallel for

```
1  #pragma omp parallel for [clause ...] newline
2  for_loop
3
```

`#pragma omp parallel for` parallelizes the execution of a single for loop over multiple threads. It corresponds to

```
1 #pragma omp parallel
2 {
3     #pragma omp for
4     for (int i = 0; i < n; i++)
5     {
6         // for loop content
7     }
8 }
9
```

```
1 #pragma omp parallel for
2 for (int i = 0; i < n; i++)
3 {
4     // for loop content
5 }
6
```

- Of course, this only makes sense if instances of the for loop are independent
- Consider

`a[i] = a[i-1] + i` //(needs `a[i-1]`)

VS.

`a[i] = 5*i + 17` //(independent of all other values in `a`)

Example

Compute $1 + 2 + \dots + n$ for some n

```
1  int main(int argc, char** argv)
2  {
3      int sum = 0;
4      int n = 5000;
5
6      #pragma omp parallel for
7      for (int i = 0; i < n; i++)
8      {
9          sum += (i+1);
10     }
11 }
12
```

Synchronization

Control how the execution of each thread proceeds relative to the other threads on the same team.

Master

A region that is to be executed only by the master thread of the team. All other threads on the team skip this section of code. There is no implied barrier associated with this directive.

```
1  #pragma omp master newline
2      structured_block
3
```

Barrier

Synchronizes all threads in the team. When a Barrier directive is reached, a thread will wait at that point until all other threads have reached that barrier. All threads then resume executing in parallel the code that follows the barrier.

```
1  #pragma omp barrier newline
2
```

Critical

Specifies a region of code that must be executed by only one thread at a time.

```
1  #pragma omp critical [name] newline
2      structured_block
3
```

Atomic

Specifies that a specific memory location must be updated atomically, rather than letting multiple threads attempt to write to it. In essence, this directive provides a mini-critical section.

```
1  #pragma omp atomic newline
2      statement_expression
3
```

And more: TASKWAIT, FLUSH, ORDERED

```
1 // Include the OpenMP header
2 #include <omp.h>
3
4 // This we only need to write output
5 #include <stdio.h>
6
7 int main(int argc, char **argv)
8 {
9     int sum = 0;
10
11     int n = 5e+6;
12
13     #pragma omp parallel for
14     for (int i = 0; i < n; i++)
15     {
16         sum += (i + 1);
17     }
18
19     printf("\n");
20     printf("Gauss says, the answer should be %i \n", (int)(n / 2) * (n + 1));
21     printf("Our answer is: %i \n", sum);
22 }
```

Fixing the race condition

```
1 // Include the OpenMP header
2 #include <omp.h>
3
4 // This we only need to write output
5 #include <stdio.h>
6
7 int main(int argc, char **argv)
8 {
9     int sum = 0;
10
11     int n = 5e+6;
12
13 #pragma omp parallel for
14     for (int i = 0; i < n; i++)
15     {
16
17         // Only one thread at a time updates the sum variable. Extremely inefficient, though.
18 #pragma omp atomic
19         sum += (i + 1);
20     }
21
22     printf("\n");
23     printf("Gauss says, the answer should be %i \n", (int)(n / 2) * (n + 1));
24     printf("Our answer is: %i \n", sum);
25 }
```

private clause

Variables in its list are to be private to each thread. A new object of the same type is declared and referenced once for each thread in the team. Private variables should be assumed to be uninitialized for each thread.

```
private(list)
```

firstprivate clause

Combines the behavior of the Private clause with automatic initialization of the variables in its list.

```
firstprivate(list)
```

shared clause

Declares variables in its list to be shared among all threads in the team. A shared variable exists in only one memory location and all threads can read or write to that address.

```
shared(list)
```

default clause

Specifies a default scope for all variables in the scope of the parallel region.

```
default (shared) or default (none)
```


Reduction

Reduction performs a reduction on the variable that appear in its list.

A private copy of each listed variable is created for each thread. At the end of the scope, the reduction operator is applied to all private copies of the variable, and the final result is written into the global shared variable.

`reduction (operator: list)`

Operators: common ones are a.o. summation(+), multiplication(*), and subtraction(-), but many more

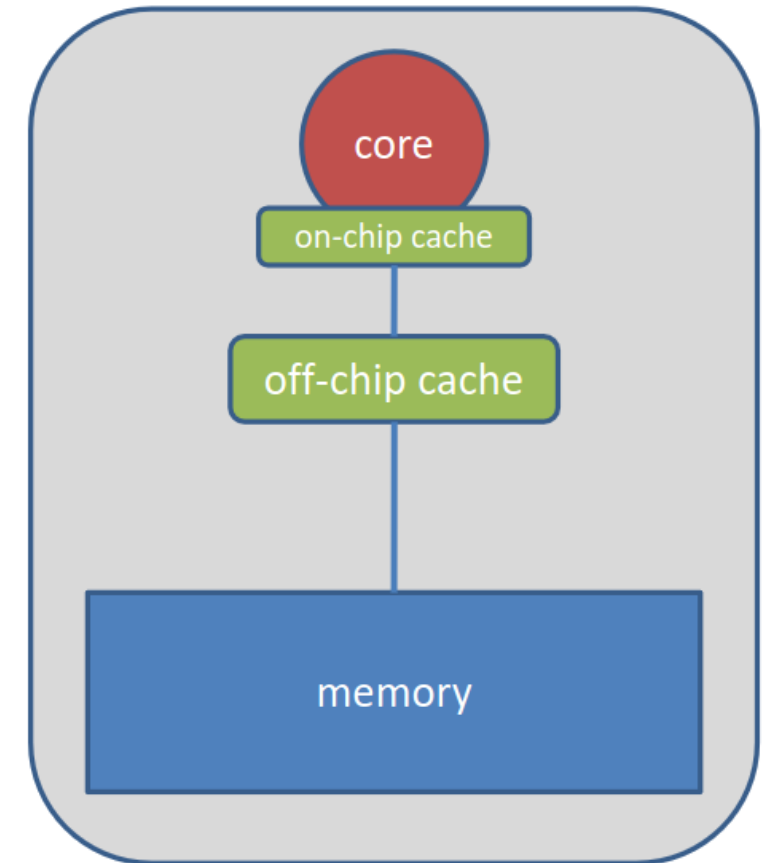
Variables: allowed are named scalar variables, no arrays or structure type variables. Variables have to be declared shared in the enclosing context.

Example:

```
1  #pragma omp parallel reduction (+ : variable)
2  #pragma omp for
3  for( ... )
4
```

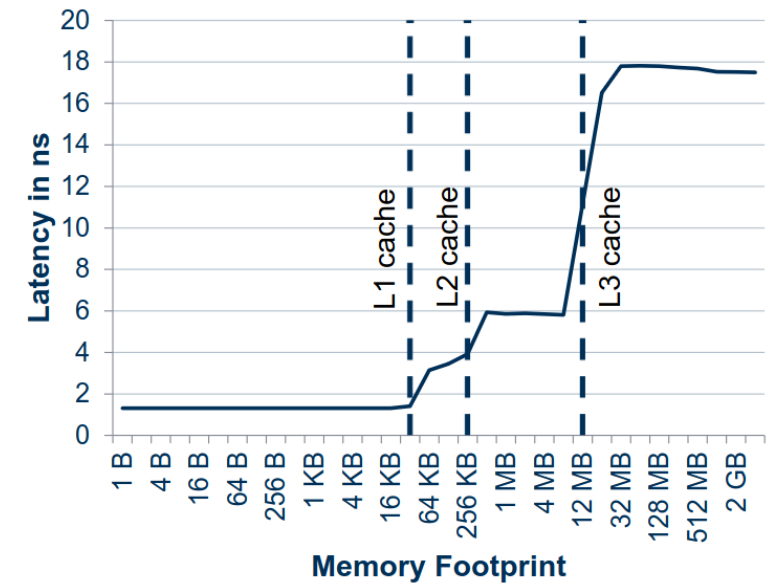
```
1 // Include the OpenMP header
2 #include <omp.h>
3
4 // This we only need to write output
5 #include <stdio.h>
6
7 int main(int argc, char **argv)
8 {
9     int sum = 0;
10
11     int n = 5e+6;
12
13     // the reduction makes sum automatically private
14     #pragma omp parallel reduction(+ : sum)
15     #pragma omp for
16     for (int i = 0; i < n; i++)
17     {
18
19         sum += (i + 1);
20     }
21
22     printf("\n");
23     printf("Gauss says, the answer should be %i \n", (int)(n / 2) * (n + 1));
24     printf("Our answer is:                %i \n", sum);
25 }
```

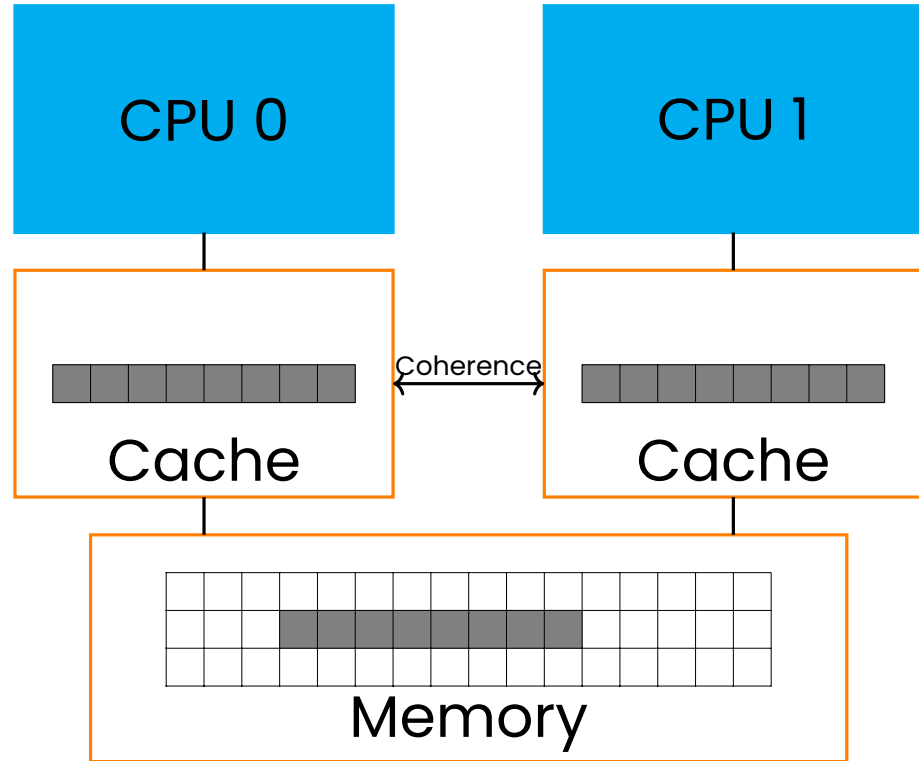
- CPUs are fast, but memory (RAM) is slow (sometimes referred to as *memory wall*)
- Cache is a small, fast memory close to the CPU
- Current CPUs have multiple levels of cache (L1, L2, L3) with different sizes and speeds
- Cache utilization is crucial for parallel performance
- Cache coherence ensures that all threads see a consistent view of memory



- CPUs are fast, but memory (RAM) is slow (sometimes referred to as *memory wall*)
- Cache is a small, but fast memory close to the CPU
- Current CPUs have multiple levels of cache (L1, L2, L3) with different sizes and speeds
- Cache utilization is crucial for parallel performance
- Cache coherence ensures that all threads see a consistent view of memory

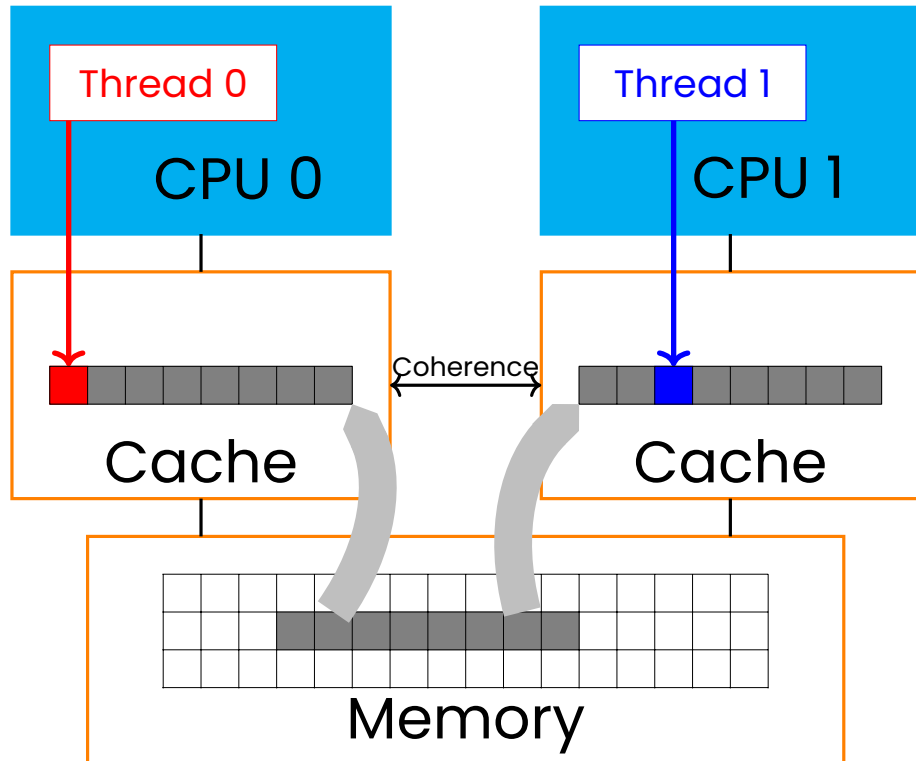
Figure 1: Latency on the Intel Westmere-EP 3.06 GHz processor



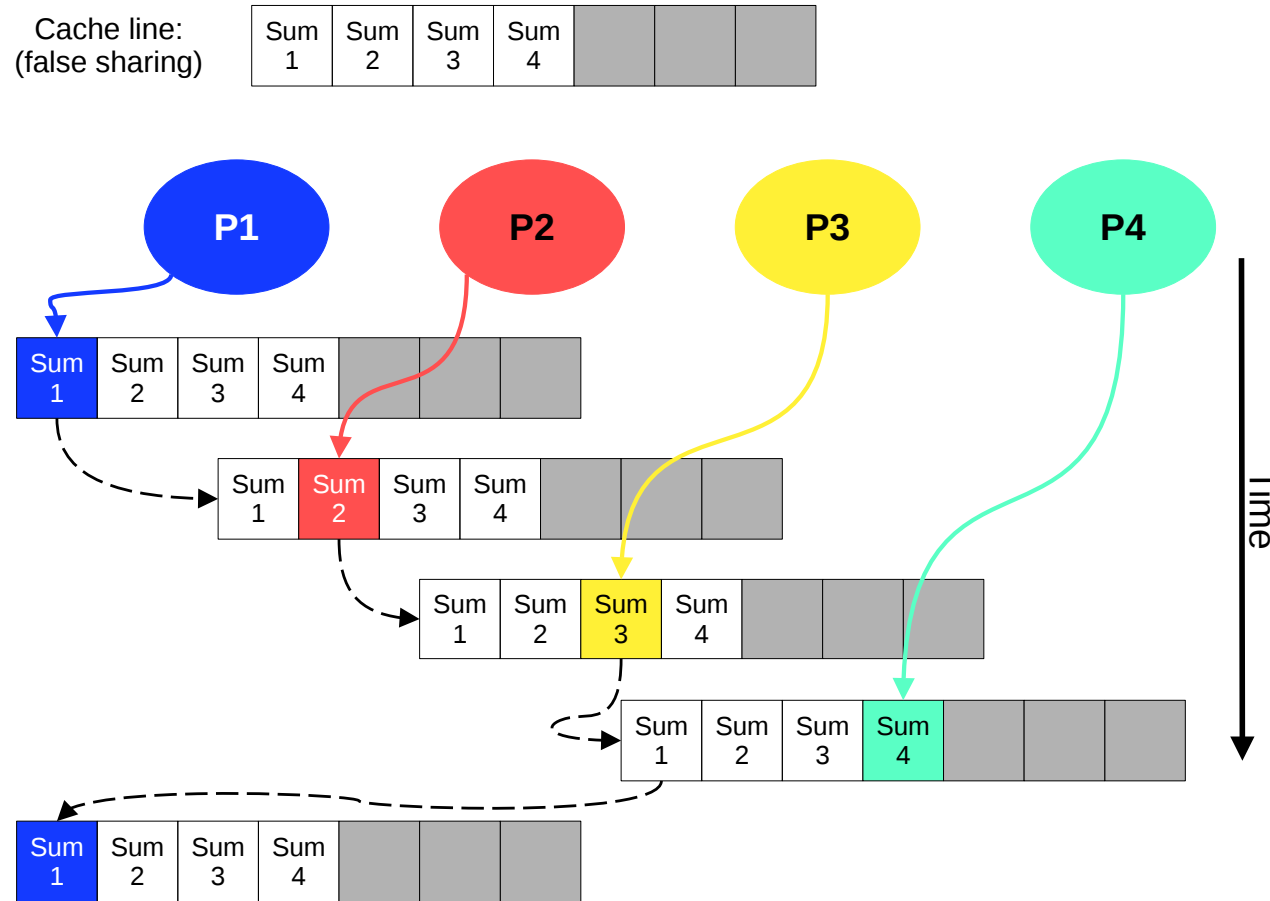


- Current used data is copied from memory into cache
- Copying is done in chunks of contiguous memory - *cache lines*
- CPU adjacent Cache holds a subset of shared memory
- If one processor changes its cache, the others need updating to maintain coherence

- This happens automatically by a Cache coherence protocol but can impact shared memory parallel performance

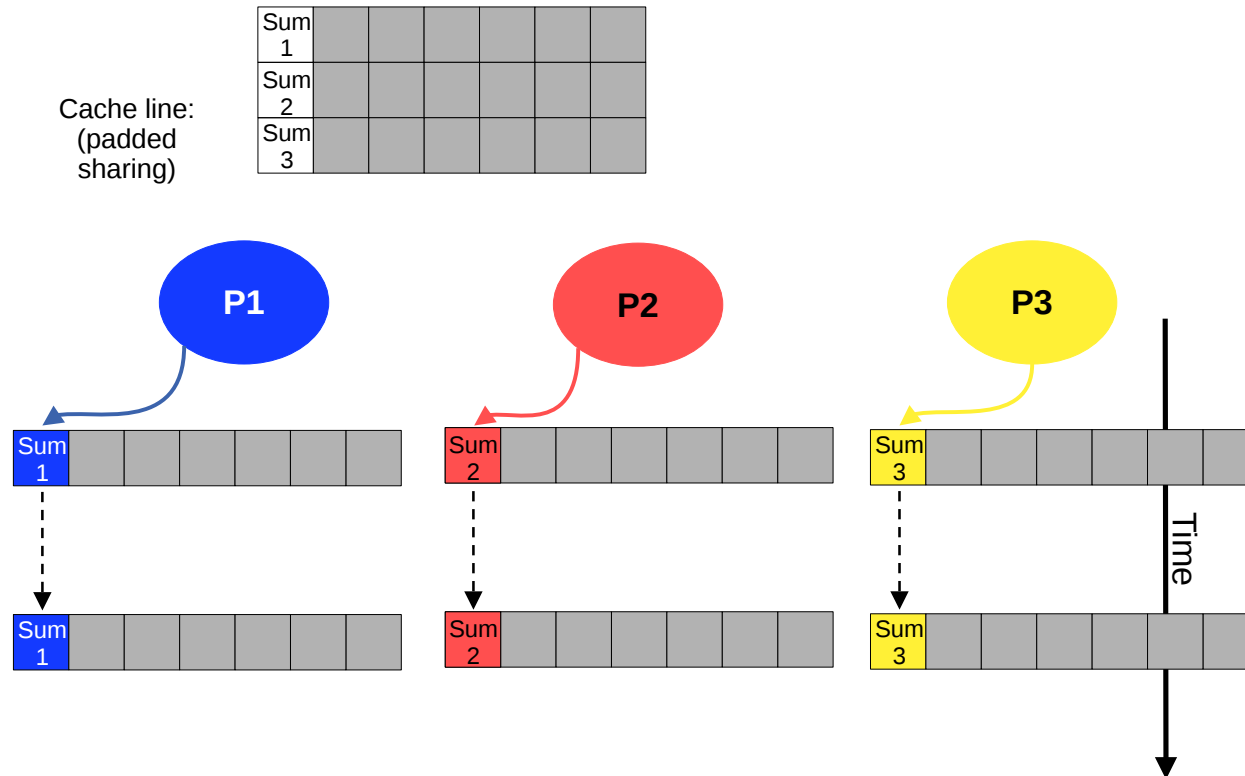


- Data is loaded into cache in “chunks” called cache lines
- A cache line holds data that is close in memory (`getconf LEVEL1_DCACHE_LINESIZE`)
- If two threads modify data in the same cache line, the Cache coherence (CC) protocol will have to update cache information very, very frequently
....this causes dramatic deterioration of performance



- Different threads use different elements, but in same cache line
- Cache line must be moved between threads for Cache coherence
- False sharing is a performance and **not** a correctness problem

```
1 #include <omp.h>
2 #include <stdio.h>
3
4 int main() {
5     int num_threads = omp_get_max_threads();
6     // Assuming an int is 4 bytes, and a cache line is typically 64 bytes, each cache line can hold 16 ints.
7     // To demonstrate false sharing, we use an array where each element is likely to be on the same cache line.
8     int data[num_threads];
9
10    // Initialize the array
11    for (int i = 0; i < num_threads; ++i) {
12        data[i] = 0;
13    }
14
15    #pragma omp parallel num_threads(num_threads)
16    {
17        int thread_id = omp_get_thread_num();
18        for (int i = 0; i < 1e7; ++i) {
19            // Each thread modifies its own unique element, but these modifications can lead to false sharing
20            data[thread_id]++;
21        }
22    }
23
24    // Print the results
25    for (int i = 0; i < num_threads; ++i) {
26        printf("data[%d] = %d\n", i, data[i]);
27    }
28
29    return 0;
30 }
```

- We *fill* the data in each thread such that it fills a full cache line
- Different threads use different elements, in **different** cache line

False sharing → Padded sharing

```
1 #include <omp.h>
2 #include <stdio.h>
3
4 struct PaddedInt {
5     int value;
6     char padding[64 - 4]; // Assuming 4 threads here
7 };
8
9 int main() {
10     int num_threads = omp_get_max_threads();
11     struct PaddedInt data[num_threads];
12
13     // Initialize the array
14     for (int i = 0; i < num_threads; ++i) {
15         data[i].value = 0;
16     }
17
18     #pragma omp parallel num_threads(num_threads)
19     {
20         int thread_id = omp_get_thread_num();
21         for (int i = 0; i < 1e7; ++i) {
22             // Each thread modifies its own unique padded element
23             data[thread_id].value++;
24         }
25     }
26
27     // Print the results
28     for (int i = 0; i < num_threads; ++i) {
29         printf("data[%d] = %d\n", i, data[i].value);
30     }
31
32     return 0;
33 }
```

You should now be familiar with the following terms:

- parallel region construct: `#pragma omp parallel`
- for directive: `#pragma omp for`
- combined parallel for: `#pragma omp parallel for`
- Synchronization: `master, barrier, critical, atomic`
- Data-sharing attributes: `private, shared, firstprivate, default`
- Reduction: `reduction (operator: list)`
- Cache coherence and false sharing

Homework

Homework: Parallelize provided matrix-vector product code using OpenMP pragmas.

Run the small example and only check for correctness (performance comes later)